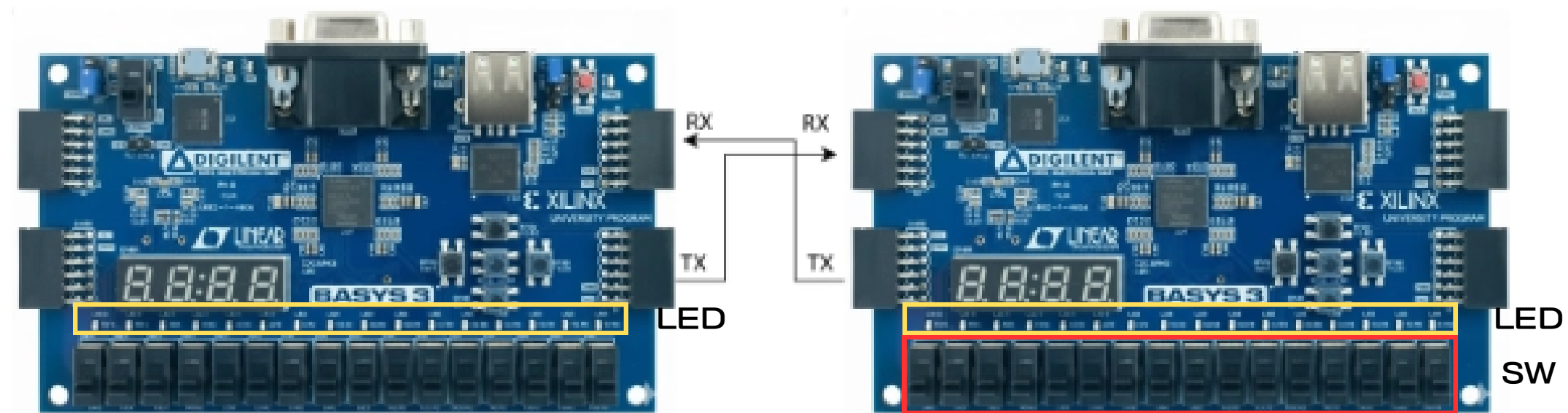


03. I2C/SPI 기반 주변장치 제어

프로젝트 개요



Board 1: Master 역할, MicroBlaze 탑재

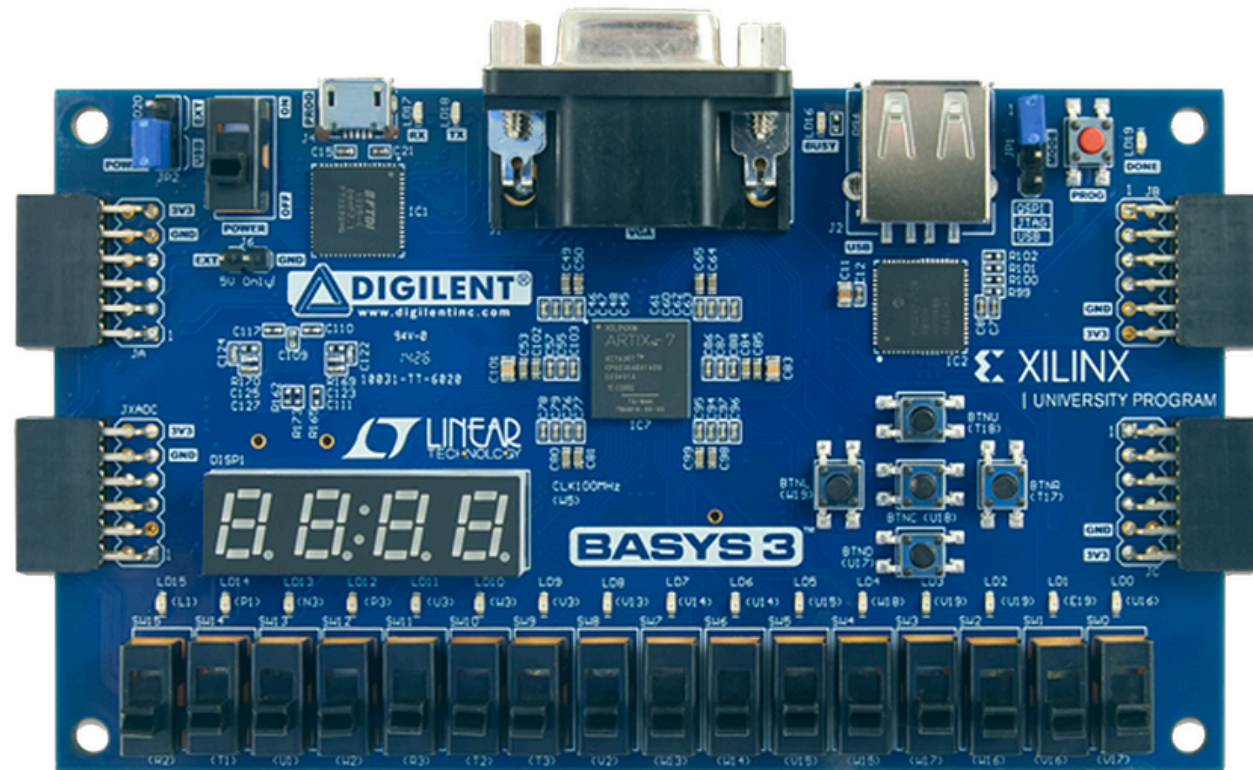
Board 2: Slave 역할, Switch 입력 발생

Board 2의 Switch 값 → Board 1의 MicroBlaze → Master와 Slave LED가 동일하게 점등

프로젝트 목적

1. MicroBlaze 기반 임베디드 시스템 구조 이해
2. VITIS 기반 Register Read/Write 제어 학습
3. I2C/SPI 통신을 이용한 보드 간 데이터 송수신 확인

개발 환경

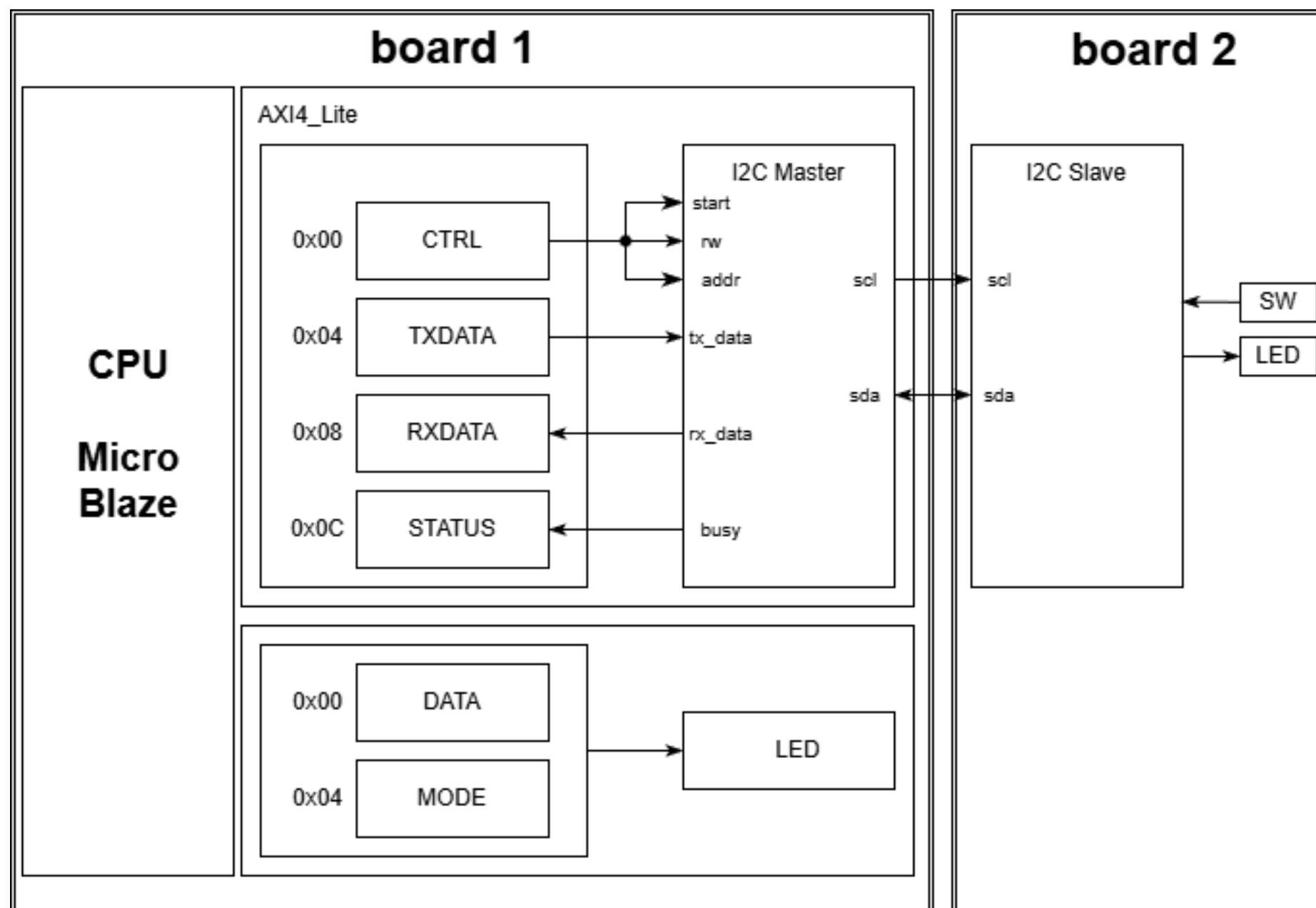


Basys3(FPGA)



Logic Analyzer

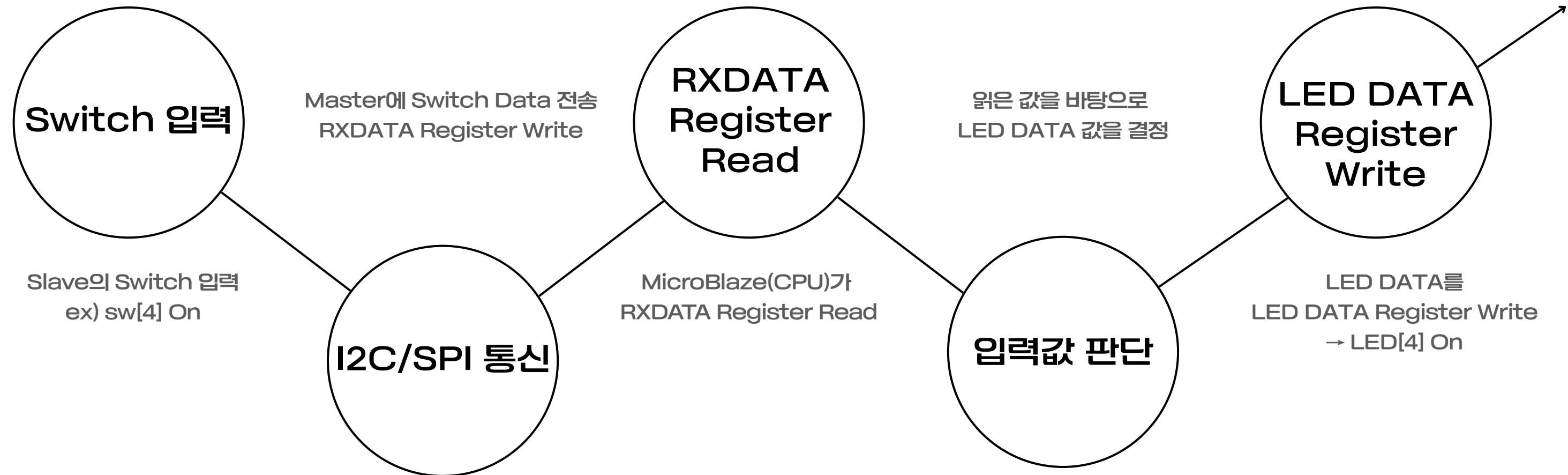
시스템 구성



Slave의 입력 스위치 값을 I2C/SPI 프로토콜을 통해 Master로 전달하고, Master는 이를 AXI4-Lite를 통해 CPU(Micro Blaze)에 전달

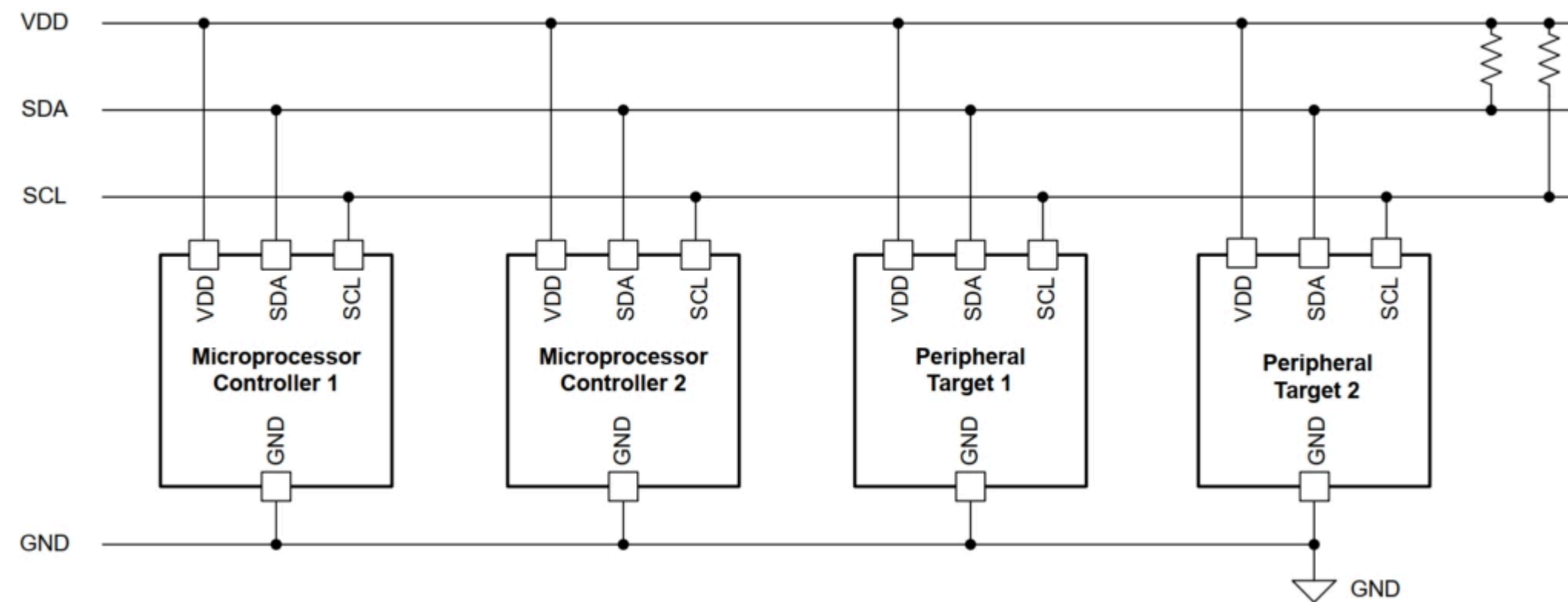
CPU는 주기적으로 I2C/SPI 트랜잭션을 발생시켜 스위치 상태를 읽어오며, 읽은 값에 따라 Master와 Slave 양측의 대응 LED를 On/Off 제어

펌웨어 관점의 제어 흐름



I2C 기반 보드 간 통신

Inter-Integrated Circuit (I2C)



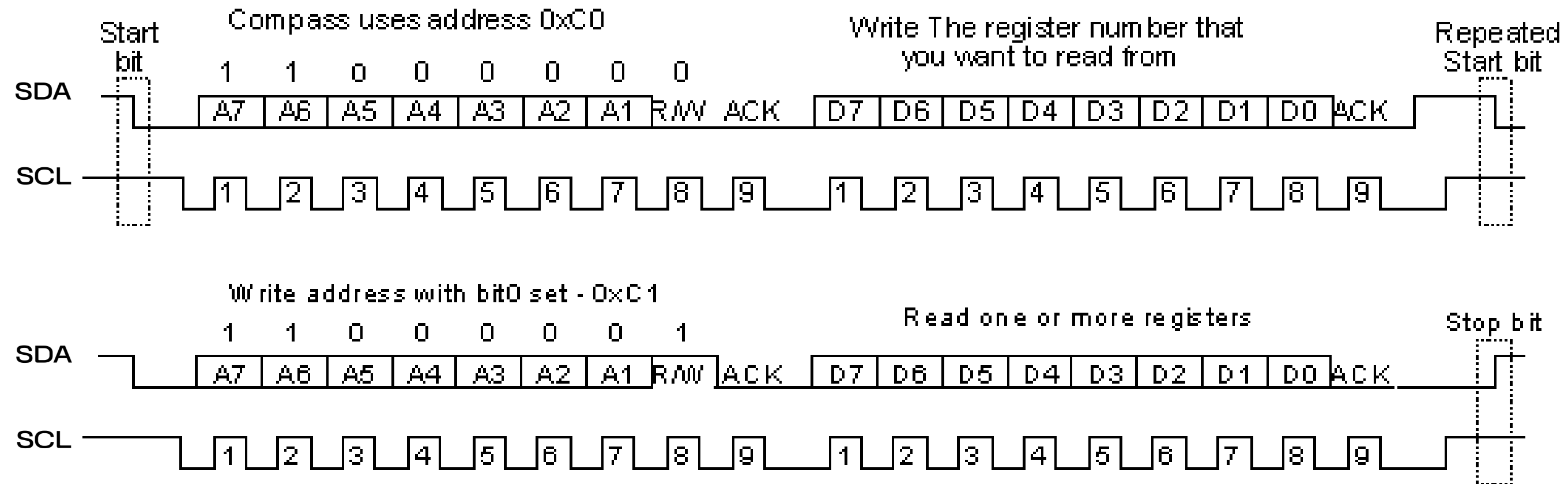
2개 라인 사용: 데이터 전송용 SDA(Serial Data)와 클록 동기용 SCL(Serial Clock) 두 선만 사용

멀티 마스터/슬레이브: 하나의 버스에 여러 개의 마스터와 슬레이브 장치를 연결 가능

주소 방식: 각 슬레이브 장치는 고유의 7bit 혹은 10bit 주소를 가지며, 마스터는 이 주소를 통해 특정 장치를 선택

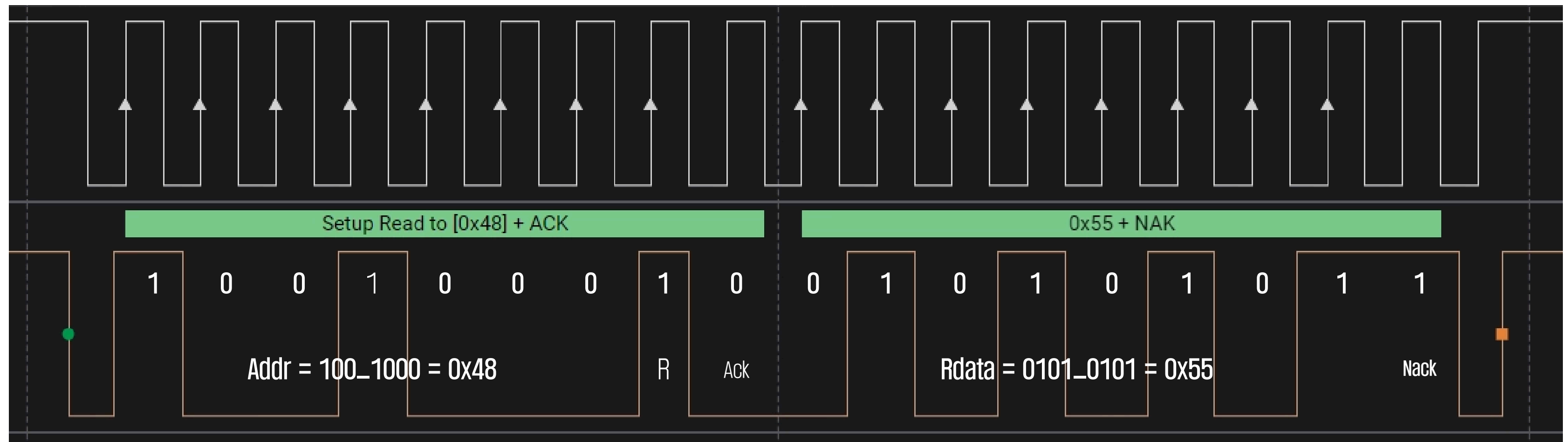
반이중방식(Half Duplex): 데이터 송신과 수신을 동시에 할 수 없으며 번갈아가며 진행

I2C 기반 보드 간 통신



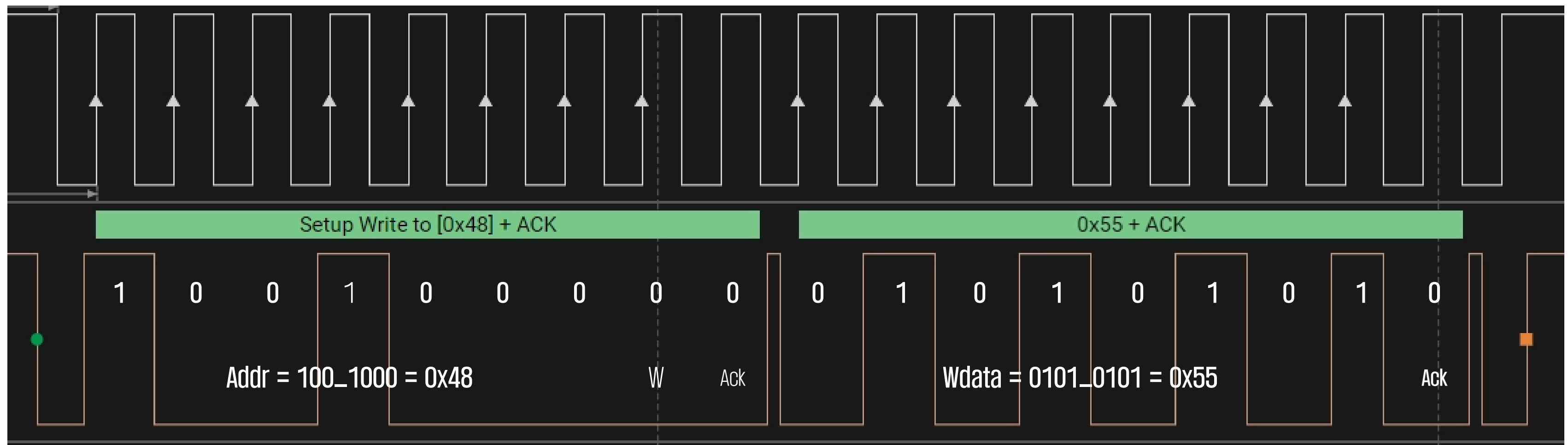
1. SCL이 High 상태에서 SDA를 High → Low로 바꾸면서 Start (SCL 생성)
2. SDA에 7-bit 주소 + 1-bit R/W 전송
3. 9번째 클럭에서 수신측이 SDA를 Low로 당기면 ACK
4. SDA를 통해 8bit 데이터 송수신
5. SCL이 High 상태에서 SDA를 Low → High로 바꾸면서 STOP (SCL HIGH)

Logic Analyzer



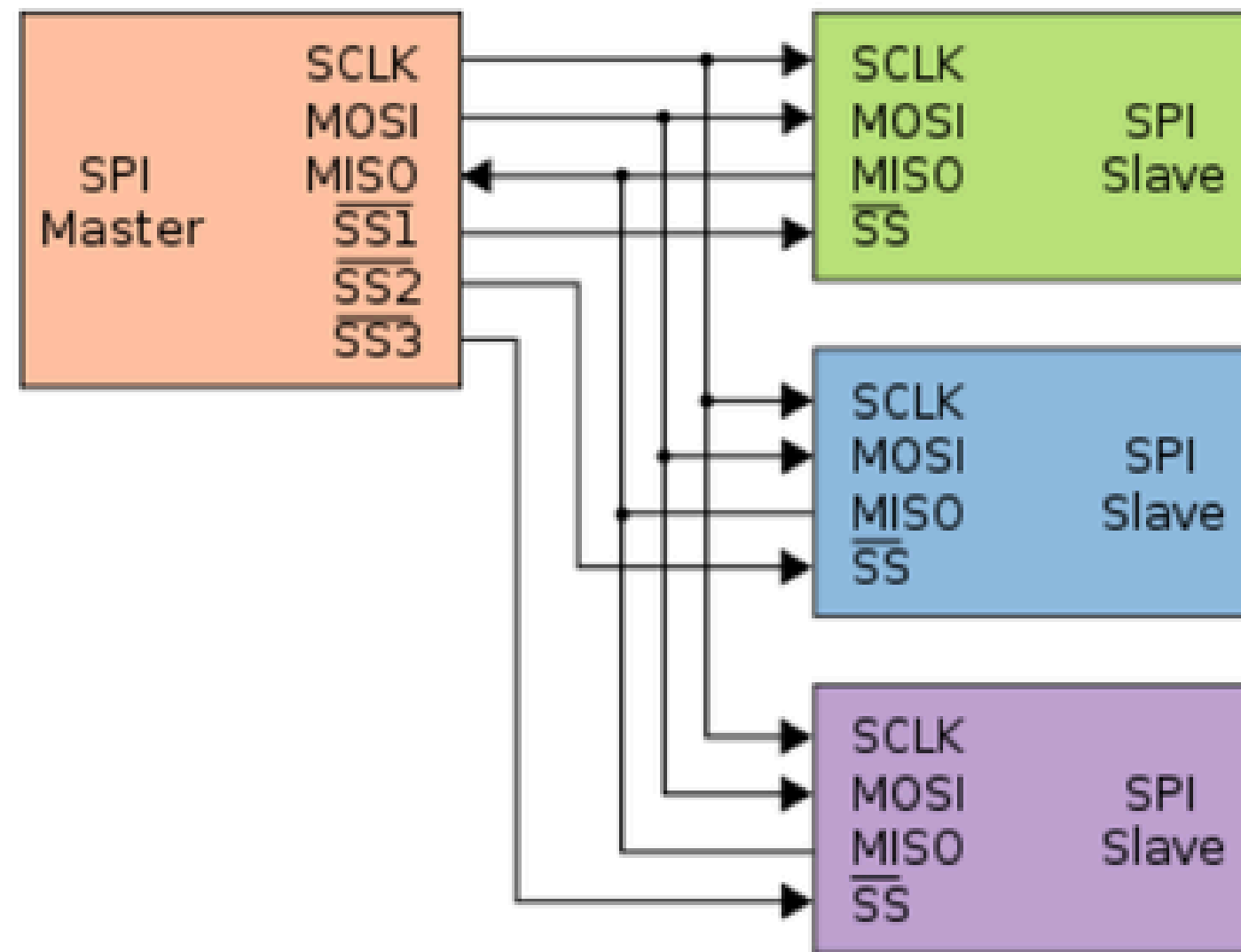
더 이상 Read를 하지 않으므로 데이터 전송 후 Nack출력

Logic Analyzer



읽은 값 0x55를 slave에 그대로 전송

SPI 기반 보드 간 통신



SCLK (Serial Clock): 데이터 동기화를 위한 클럭 신호 (마스터가 출력)
MOSI (Master Out Slave In): 마스터에서 슬레이브로 데이터를 보낼 때 사용
MISO (Master In Slave Out): 슬레이브에서 마스터로 데이터를 보낼 때 사용
SS/CS (Slave Select/Chip Select): 통신할 특정 슬레이브를 선택하는 신호

Serial Peripheral Interface (SPI)

마스터-슬레이브 구조를 사용하는 4선식

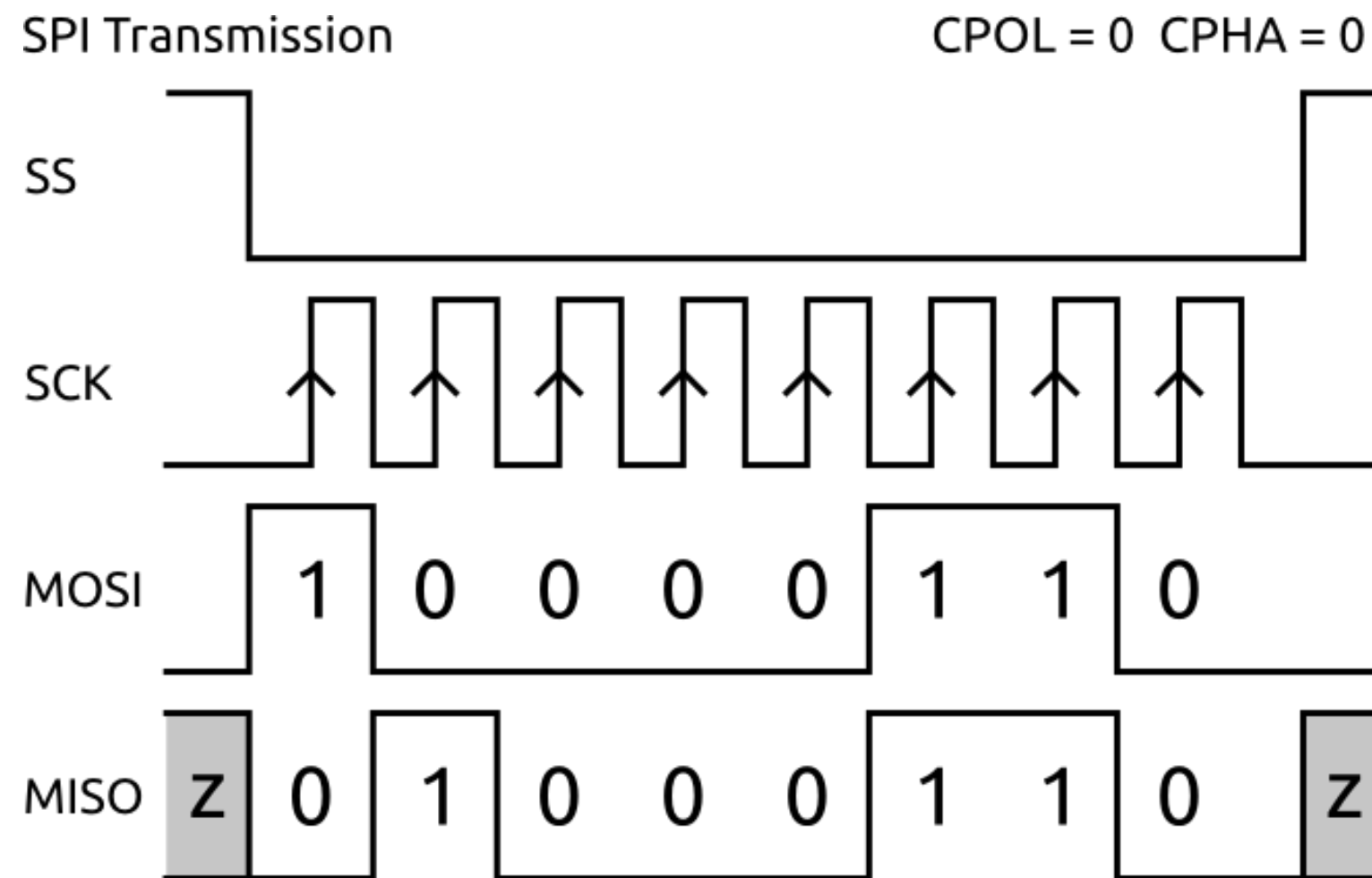
동기 통신 프로토콜

데이터를 보내는 동시에 받을 수 있어 속도가 빠름

단일 마스터가 여러 슬레이브를 제어 가능

CPOL과 CPHA에 따라 4가지 모드로 작동

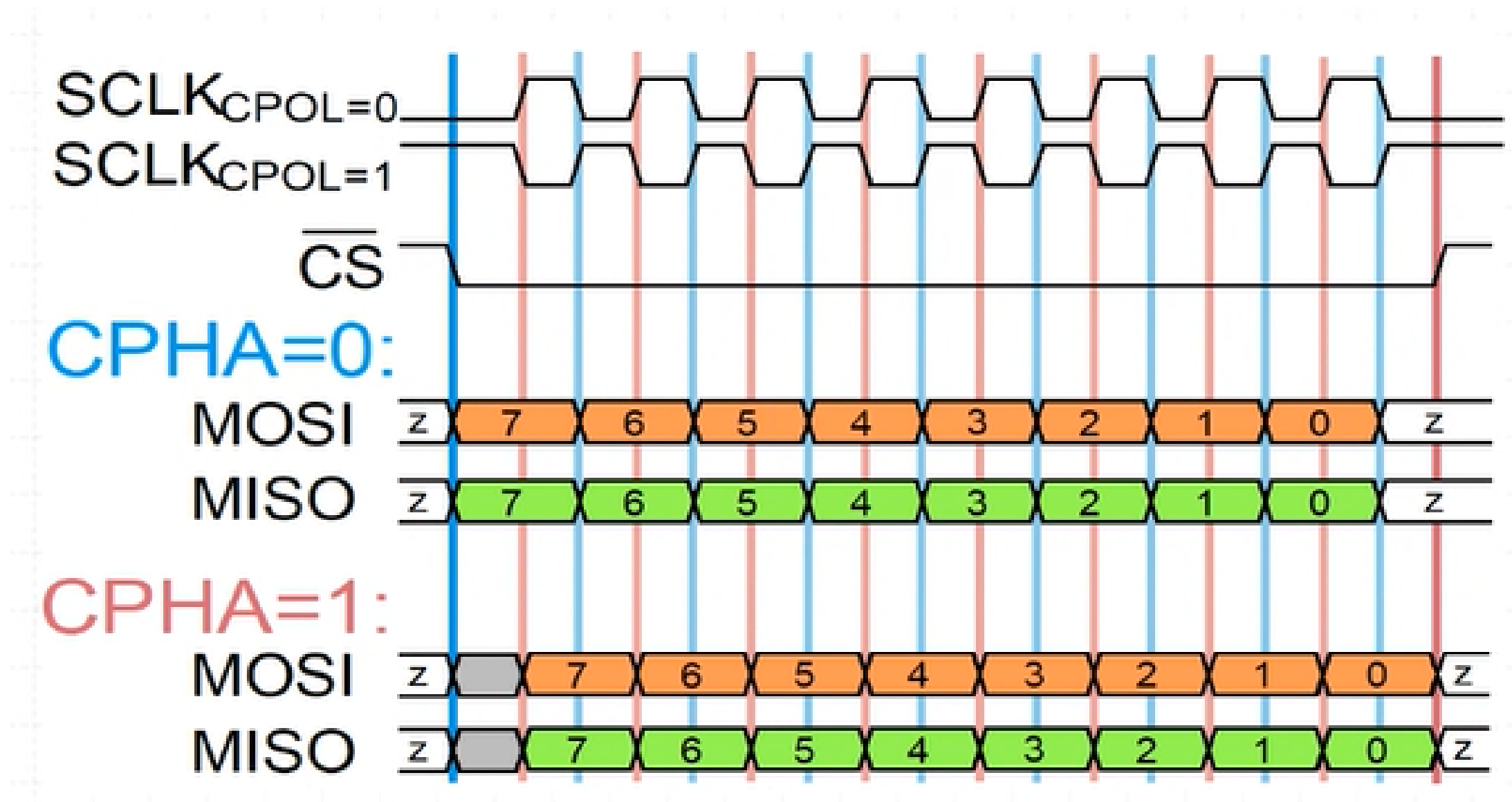
SPI 기반 보드 간 통신



1. 마스터가 특정 슬레이브의 SS/CS 핀을 Low로 내려 통신 활성화
2. 마스터가 SCLK를 생성하고, MOSI라인을 통해 데이터를 전송하는 동시에 MISO라인으로 데이터를 수신
3. 데이터 전송이 완료되면 SS/CS핀을 High로 올려 통신 종료

장점: 고속 전송, 단순한 구조, 양방향 동시 통신, 1 : N 통신
단점: 슬레이브 증가시 핀 수가 많아짐, 장거리 통신 부적합

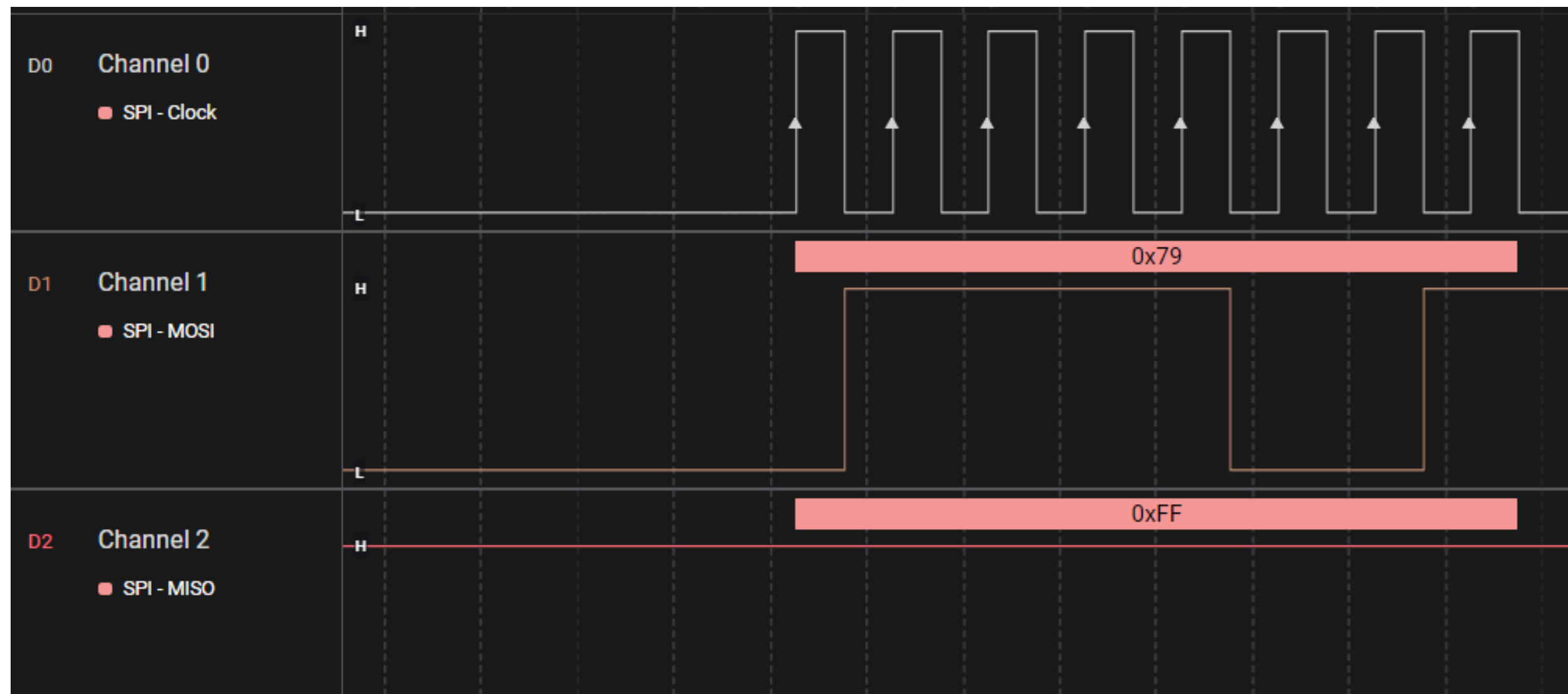
SPI 기반 보드 간 통신



CPOL (Clock Polarity) 클럭의 IDLE 상태: 0(LOW), 1(HIGH)

CPHA (Clock Phase) 데이터 샘플링 타이밍: 0(rising edge), 1(falling edge)

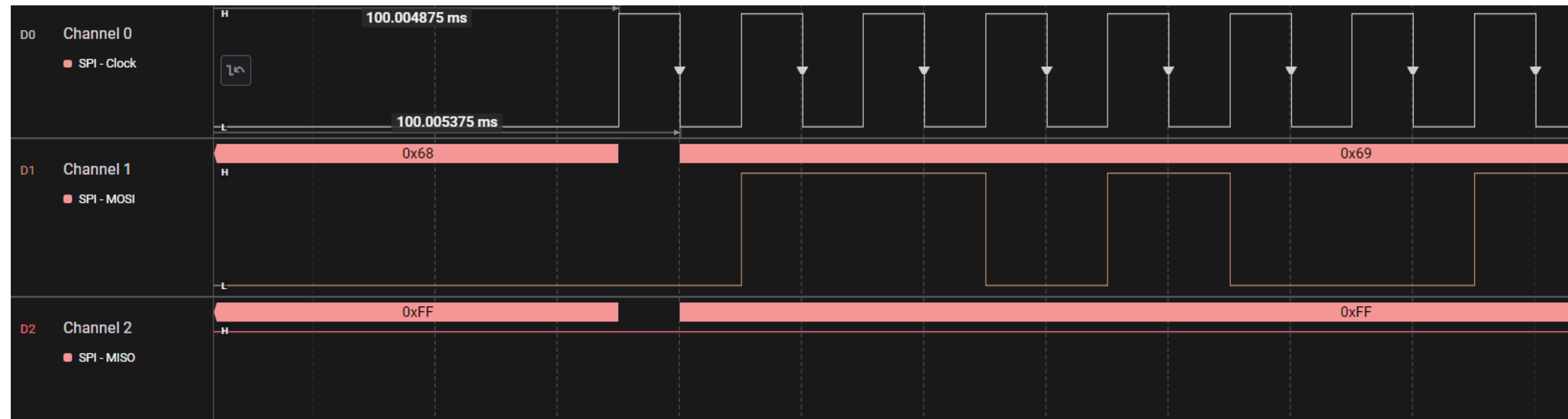
Logic Analyzer



Mode0 (CPOL=0, CPHA=0)

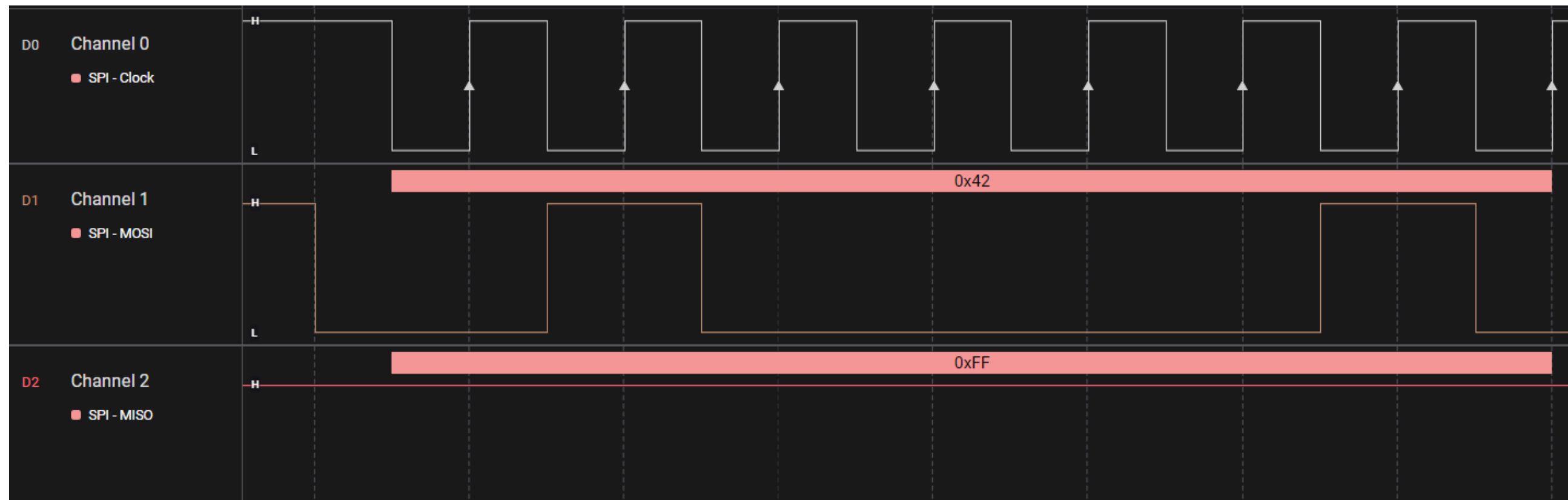
Master 단독테스트(MISO X)

Logic Analyzer



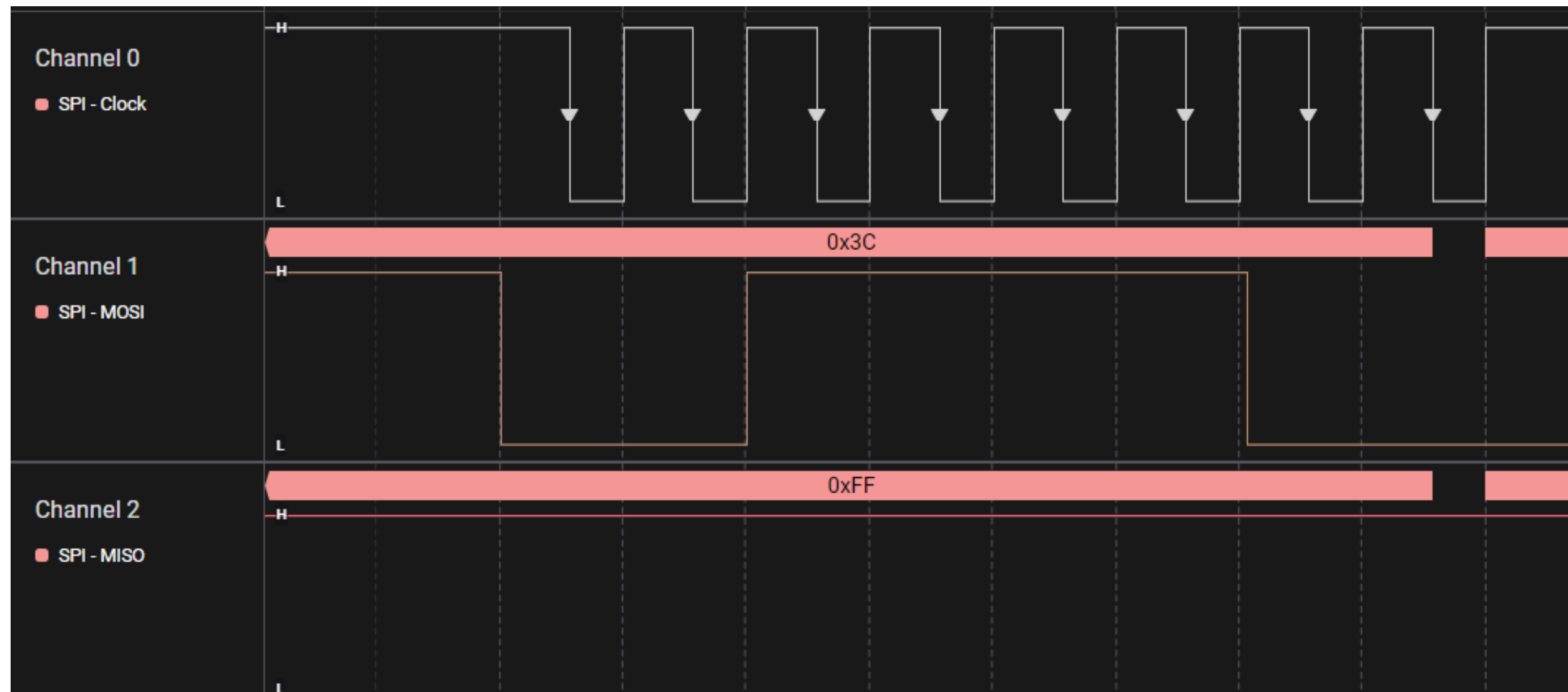
Mode1 (CPOL=0, CPHA=1)
Master 단독테스트(MISO X)

Logic Analyzer



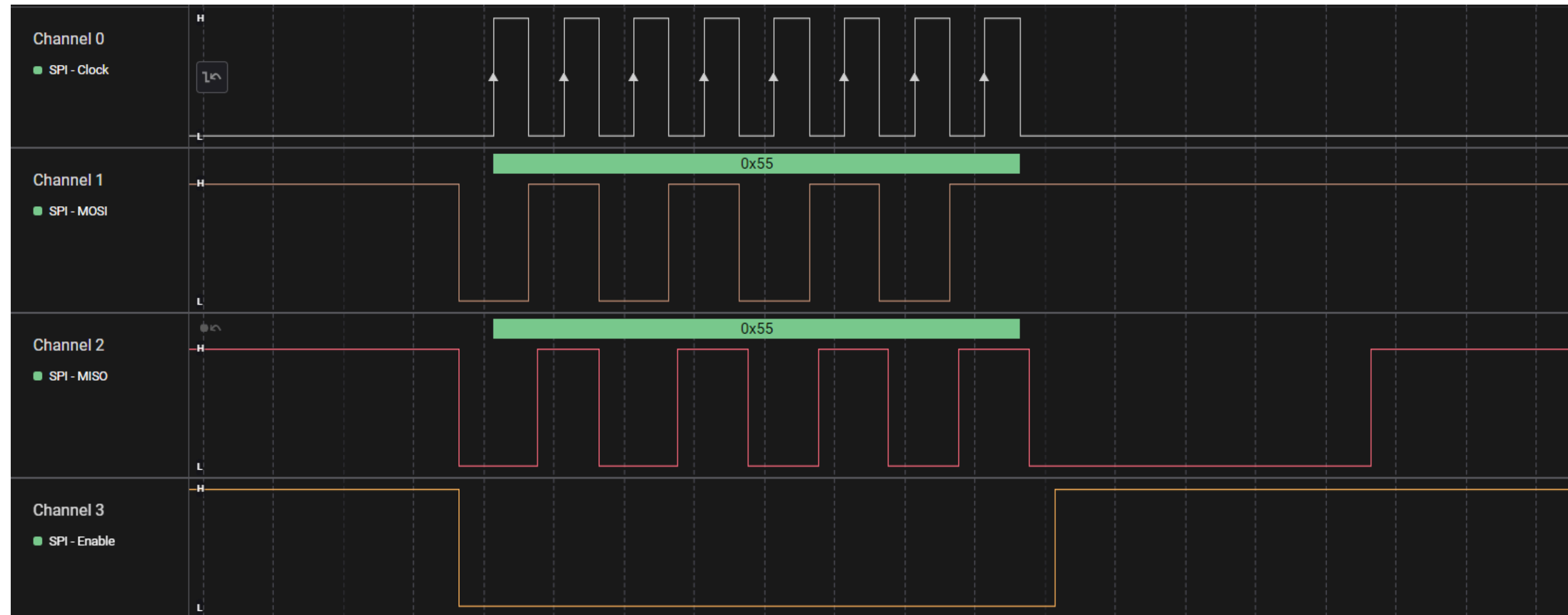
Mode2 (CPOL=1, CPHA=0)
Master 단독테스트(MISO X)

Logic Analyzer



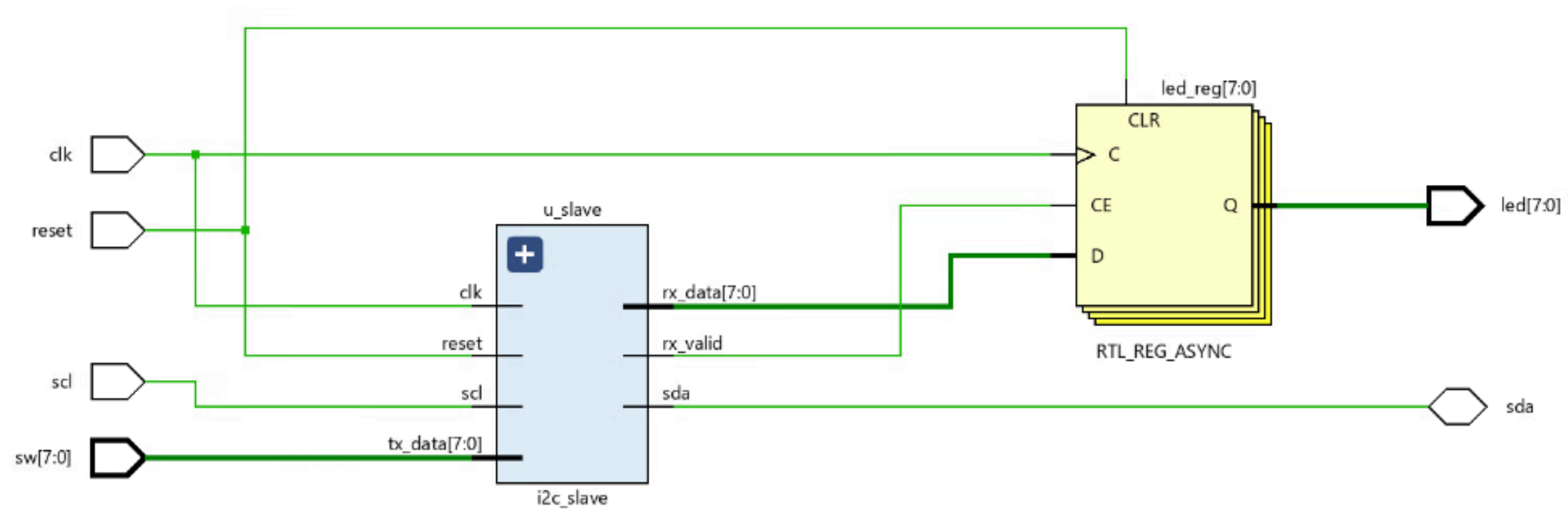
Mode3 (CPOL=1, CPHA=1)
Master 단독테스트(MISO X)

Logic Analyzer

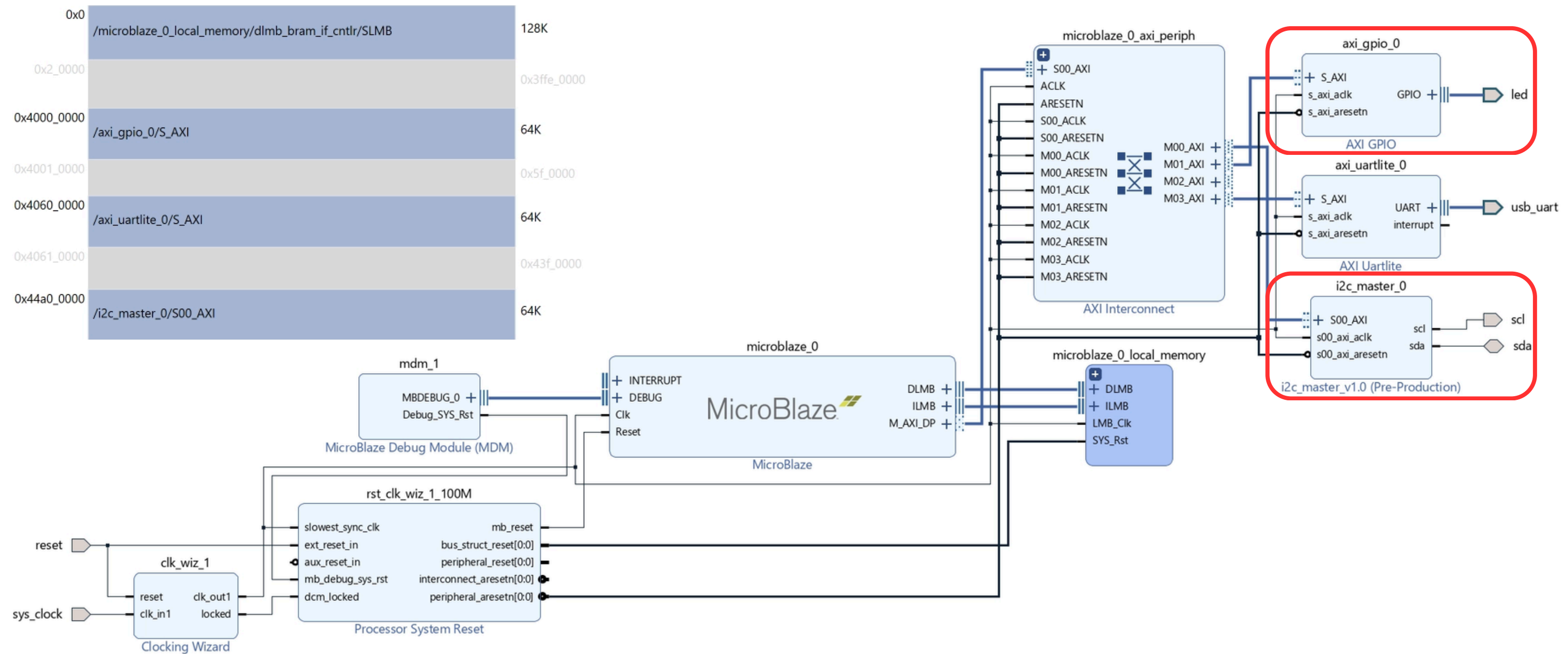


Master <-> Slave 연결 후 Mode0에서 Test
MOSI, MISO 둘 다 확인

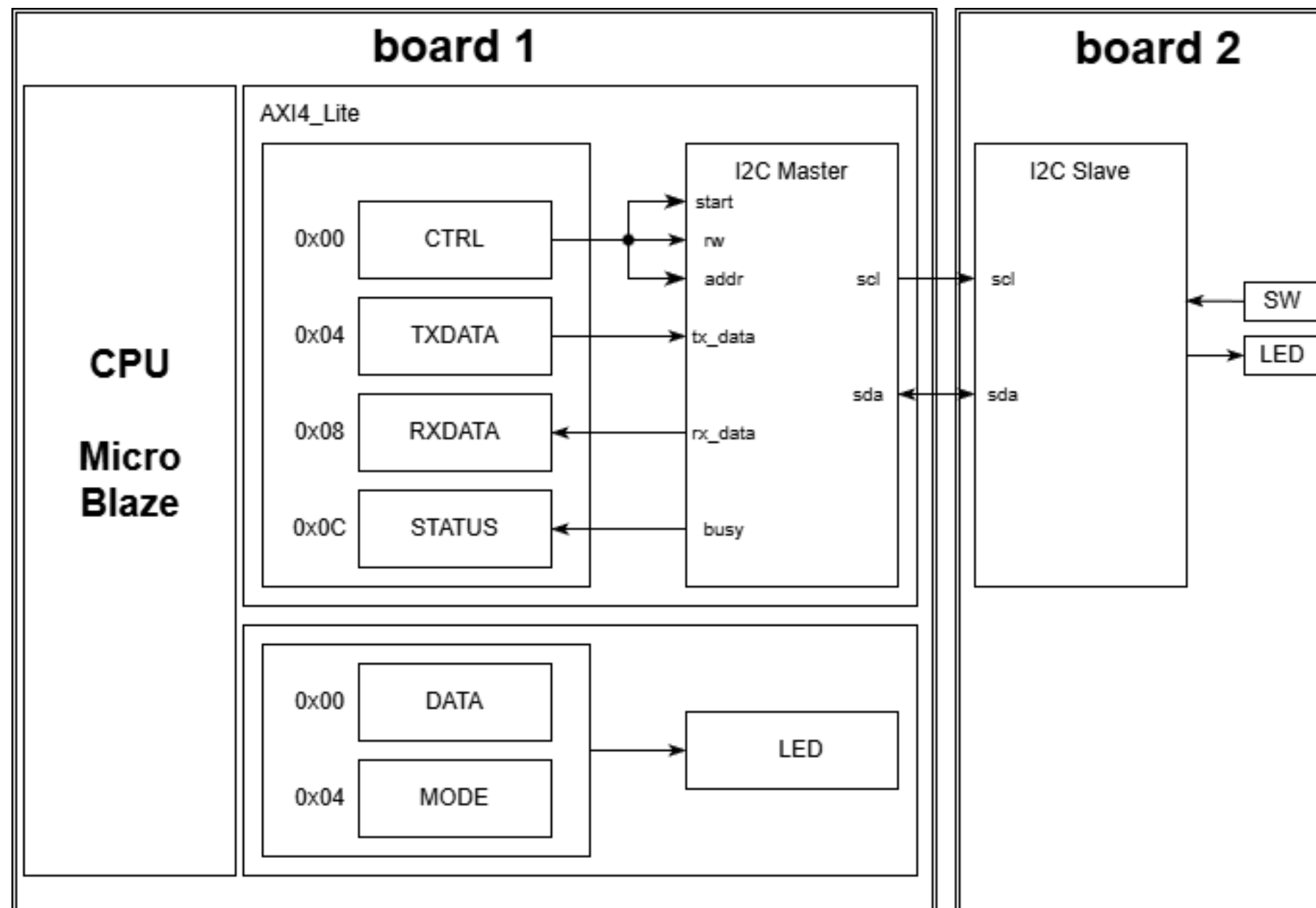
Slave 구조



Master 구조



Register 제어



I2C/SPI Register 제어

Address	Register	펌웨어 관점 역할
0x00	CTRL	Start_trigger, R/W, Address
0x04	TXDATA	tx_data
0x08	RXDATA	rx_data
0x0C	STATUS	busy

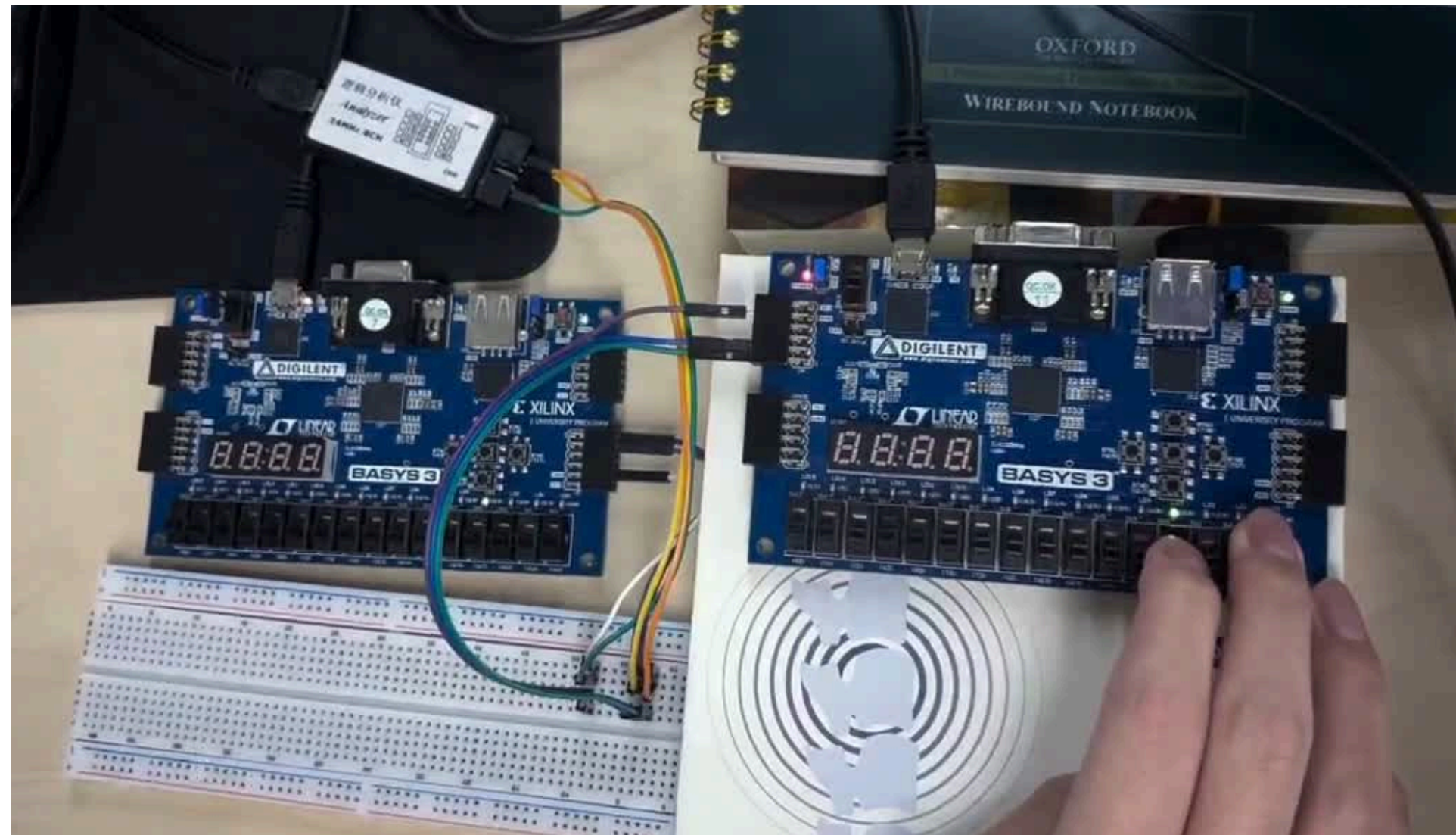
Base Address: 0x44A0_0000

GPIO Register 제어

Address	Register	펌웨어 관점 역할
0x00	LED DATA	LED 출력값 설정
0x04	LED MODE	LED 출력 모드 설정

GPIO 모듈을 사용해 LED를 제어하기 때문에 LED MODE를 OUTPUTMODE로 설정
Base Address: 0x4000_0000

동작 영상

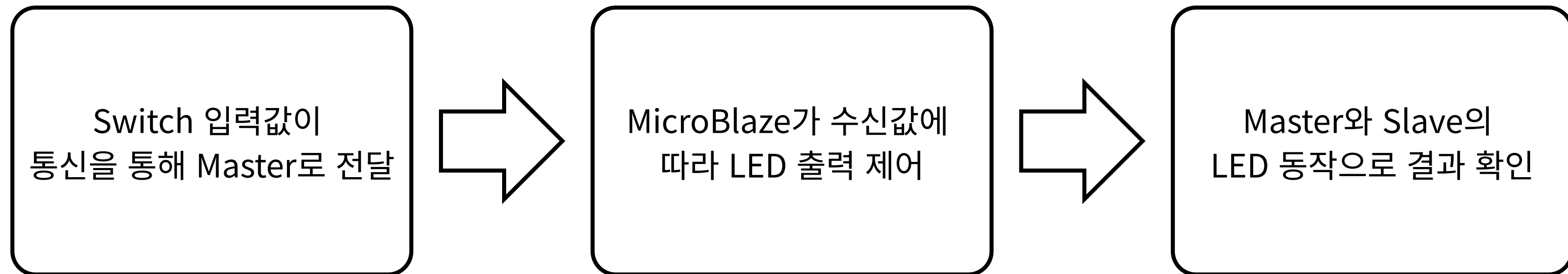


좌: Master

우: Slave

Slave의 SW를 조작하여 Master와 Slave의 LED가 점등되는 것을 확인

결과 정리



펌웨어 개발을 통해 얻은 역량

Register 제어

하드웨어 Register Read/Write를 통한 제어 방식 이해

데이터 흐름 설계

입력 → 처리 → 출력으로 이어지는 펌웨어 구조 경험

디버깅 및 검증

LED 출력, UART 로그, 파형 분석을 통한 동작 확인